1 openai接口

1.1 非流式调用

- 代码

```
from openai import OpenAI
import openai

openai.api_key = 'sk-RZH5BgtgdDazdwEZD4D50cC6Cd984a8dA838B482918d8e75'
openai.base_url = 'https://api.xty.app/v1'

def get_completion(prompt, model="gpt-3.5-turbo"):
    client = OpenAI(api_key=openai.api_key,
                    base_url=openai.base_url
                    )
    messages = [{"role": "user", "content": prompt}]

    response = client.chat.completions.create(
        model=model,
        messages=messages,
        stream=False
    )
    return response.choices[0].message.content

prompt = '你好，请介绍下你自己'
get_completion(prompt, model="gpt-3.5-turbo")

```

1.2 流式调用

- 代码

```
import openai
import sys
from openai import OpenAI

# 设置你的 API 密钥
openai.api_key = 'sk-RZH5BgtgdDazdwEZD4D50cC6Cd984a8dA838B482918d8e75'
openai.base_url = 'https://api.xty.app/v1'

def get_stream_completion(prompt, model="gpt-3.5-turbo"):
    # 定义模型
    client = OpenAI(api_key=openai.api_key
```

```
                        , base_url=openai.base_url
                        )
    # 定义消息
    messages = [{"role": "user", "content": prompt}]

    def stream_response():
        response = client.chat.completions.create(
            model=model,
            messages=messages,
            stream=True  # 启用流式输出
        )

        for chunk in response: # 这段是官网提供的
            if chunk.choices[0].delta.content is not None:
                # print(chunk.choices[0].delta.content, end="")
                yield chunk.choices[0].delta.content

    return stream_response()

prompt = "请告诉我关于brave knight的故事"
response_generator = get_stream_completion(prompt, model="gpt-3.5-turbo")

for response_part in response_generator:
    print(response_part,end='')
```

1.3 多轮对话

- 代码

```
import openai
from openai import OpenAI

# 设置你的 API 密钥
openai.api_key = 'sk-RZH5BgtgdDazdwEZD4D50cC6Cd984a8dA838B482918d8e75'
openai.base_url = 'https://api.xty.app/v1'

client = OpenAI(api_key=openai.api_key
                , base_url=openai.base_url
                )
history = [
    {"role": "system",
     "content": "你是 gpt-3.5，由 openai 提供的人工智能助手，你更擅长中文和英文的对话。你会为用户提供安全，有帮助，准确的回答。同时，你会拒绝一切涉及恐怖主义，种族歧视，黄色暴力等问题的回答。"}
]

def chat(query, history):
```

```
    history.append({
        "role": "user",
        "content": query
    })
    completion = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=history,
        temperature=0.3,
    )
    result = completion.choices[0].message.content
    history.append({
        "role": "assistant",
        "content": result
    })
    return result

print(chat("你好我的名字叫孟智超", history))
print(chat("请问你还记得我的名字吗", history))

"""输出"""
你好，孟智超！很高兴认识你，有什么我可以帮忙的吗？

当然记得，你的名字是孟智超！有什么我可以帮助你的吗？
```

2 lora

2.1 什么是LoRA

- LoRA 全称为 Low-Rank Adaptation，是一种用于高效调整预训练模型的方法。它的核心思想是通过添加低秩矩阵来近似表示模型的权重更新，而不是直接更新原始的高维权重矩阵。这样可以在不显著增加计算和存储成本的情况下，实现模型的快速调整和适应

2.2 LoRA 的基本原理

- LoRA 依赖于以下两个核心概念
    1. **低秩矩阵**：在矩阵分解中，低秩矩阵是可以通过较少的参数来逼近高秩矩阵的。LoRA 使用低秩矩阵来替代模型中的权重矩阵更新，从而减少计算复杂度。
    2. **冻结原始权重**：在LoRA中，预训练模型的原始权重是冻结的，即不在训练过程中更新。相反，LoRA 添加了一组新的低秩矩阵，并仅更新这些低秩矩阵的参数。这种方法有效降低了训练过程中的参数量。
- 原理图（论文中）

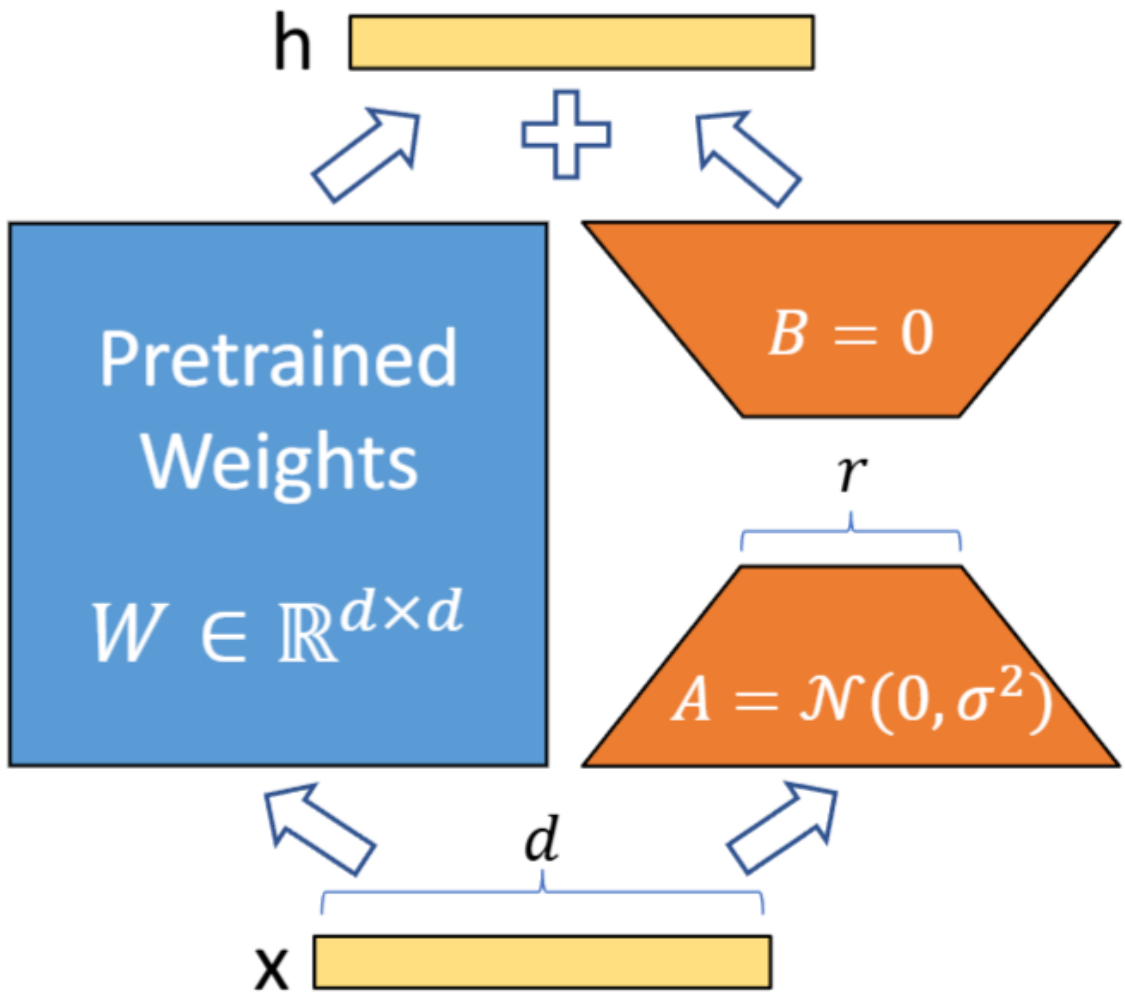

Figure 1: Our reparametrization. We only train A and B.

原模型

$$h = W\mathrm{x} \tag{1}$$

lora模型

$$h = W\mathrm{x} + AB\mathrm{x} \tag{2}$$

合并lora层

$$W' = W + AB \tag{3}$$

其中A矩阵使用高斯分布来初始化，B矩阵使用0矩阵来初始化

2.3 LoRA 的优点

- LoRA拥有几个关键优势
    1. **模块化共享**：预训练模型可以共享并用于构建许多用于不同任务的小型LoRA模块。我们可以通过替换矩阵A和B来冻结共享模型并有效地切换任务，从而显着降低存储需求和任务切换开销。
    2. **高效训练**：使用自适应优化器时，LoRA 使训练更加高效，并将硬件进入门槛降低了多达 3 倍，通过低秩矩阵来更新模型权重，显著减少了训练参数的数量，从而降低了计算和存储成本。，因为我们不需要计算梯度或维护大多数参数的优化器状态。相反，我们只优化注入的、小得多的低秩矩阵。
    3. **灵活性**：LoRA 可以应用于不同类型的神经网络，包括卷积神经网络（CNNs）和 Transformer 等，从而具备广泛的适用性。
    4. **易于集成**：由于 LoRA 不需要修改原始模型的架构，因此它可以很容易地集成到现有的深度学习框架中，如 TensorFlow 和 PyTorch。